

Implementing Classic Subword Tokenization Algorithms: BPE and WordPiece

Building the engines of modern language models from scratch

Mohamed Ibrar UL HASSAN · Temidayo James AFOLABI · Lama YASSIN

Supervisor: Professor Marie CANDITO

NLP Project - Master 1, Université Paris Cité

Academic Year 2025–2026

Abstract

This report documents the from-scratch implementation and scientific comparison of two classic subword tokenization algorithms: Byte Pair Encoding (BPE) and WordPiece. We implement both in naive and efficient (“fast”) forms, explain the theory behind them in a pedagogical way, and resolve tokenization ambiguity with principled strategies. Using a German Wikipedia corpus, we run five experiments measuring training speed, the effect of a frequency threshold on WordPiece, the effect of vocabulary size on tokenization length, the resulting attention-matrix cost, and morphological alignment against a hand-built gold standard. We find that our fast implementations turn linear-in-corpus training time into near-constant time; that the two algorithms diverge because BPE optimizes frequency while WordPiece optimizes informativeness; and that, on German, BPE’s frequency criterion aligns with morphology almost for free (0.91 morpheme recall) whereas WordPiece requires a frequency threshold to approach it and only at the cost of a saturated vocabulary. We also report, transparently, the two genuine bugs we found and fixed during the project.

Contents

Abstract.....	2
Contents	3
Key Terminology	5
The token hierarchy	5
Core terms	5
1. Introduction	6
1.1 What this report contains	7
2. Theoretical Part	7
2.1 The task: turning text into numbers	7
2.1.1 Three ways to choose a token	8
2.1.2 Two phases: training and tokenizing.....	8
2.2 BPE: the frequency method.....	9
2.2.1 Training BPE, step by step	9
2.2.2 Tokenizing with BPE	9
2.3 WordPiece: the information method	10
2.3.1 The score: frequency is not enough.....	10
2.3.2 Training and tokenizing WordPiece.....	10
2.4 Resolving ambiguity	11
2.5 Advantages and disadvantages	12
3. Experiments and Results	12
3.1 Resources: corpus and gold standard	12
3.2 The fast implementations	13
3.3 Experiment 1: training speed (naive vs fast)	13
3.4 Experiment 2: the WordPiece frequency threshold	14
3.5 Experiment 3: vocabulary size and tokenization length.....	15
3.6 Experiment 4: tokenization length and attention cost	16
3.7 Experiment 5: morphological evaluation	17
3.8 Automatic cross-check with spaCy.....	19
3.9 Problems encountered and how we solved them	19
3.9.1 The stale-heap bug in fast WordPiece	20
3.9.2 The missing training cells	20
3.9.3 The frequency-threshold trade-off	20
4. Computer-Science Part	21
4.1 Code organisation	21
4.2 Implementation choices	21

4.3 Problems encountered and their solutions	21
5. User Manual.....	22
5.1 Requirements	22
5.2 Running the project.....	22
5.3 Using the algorithms directly	22
6. Discussion, Limitations and Conclusion.....	23
6.1 What we found	23
6.2 Limitations	23
6.3 Conclusion.....	23
7. References	25

Key Terminology

To ensure our report and presentation are consistent, we fix the meaning of the central terms here and use them in exactly this sense throughout. Where the field uses several words for the same idea, we commit to one.

The token hierarchy

We distinguish clearly between a kind of unit and an occurrence of that unit, for both whole words and subword pieces.

Term	Meaning	Example
Word type	A distinct word, counted once regardless of how often it appears.	“the” is one word type
Word token	A single occurrence of a word in the text.	“the” appearing 5 times = 5 word tokens
Subword type	A distinct subword piece in the vocabulary, counted once.	“##en” is one subword type
Subword token	A single occurrence of a subword piece in tokenized text.	each use of “##en” is one subword token

In short: a type is an entry in the catalogue; a token is one appearance of it in running text. When we report “tokens per sentence,” we mean subword tokens, occurrences, not distinct types.

Core terms

Term	Definition as we use it
Tokenizer	The program that cuts text into units and, in training, learns which units to use.
Vocabulary	The full set of subword types the tokenizer knows. Its size is a parameter we choose.
Merge / merge rule	One step that glues two adjacent pieces into a new piece. BPE stores these as an ordered list.
Training	The one-off phase that reads a corpus and learns the vocabulary (and, for BPE, the merge rules).
Tokenizing	The everyday phase that applies a learned vocabulary to new text.
Naive / fast	Two implementations of the same algorithm: naive is simple but slow; fast uses efficient data structures.
Corpus	The body of text used for training. Ours is a slice of German Wikipedia.

Term	Definition as we use it
OOV (out-of-vocabulary)	A word the tokenizer has never seen. Subword tokenization avoids hard OOV failures by falling back to smaller pieces.
## prefix	WordPiece’s mark on a piece that continues a word rather than starting it, e.g. “spiel” + “##en”.
Morpheme	The smallest meaningful unit of a word that a linguist would identify, e.g. “un”, “break”, “ability”.
Gold standard	Our hand-built reference of correct morpheme splits, used to evaluate the two algorithms.
Morpheme recall	The fraction of expected morphemes that appear among a tokenizer’s output pieces.
Exact match	Whether a tokenizer’s split is exactly the expected morpheme split, with nothing added or missing.
min_pair_freq	Our tunable threshold for WordPiece: ignore any pair occurring fewer than this many times when choosing a merge.
Saturation	When a fixed threshold blocks all remaining merges, so the vocabulary stops growing below its target size.

Two conventions on phrasing. We say a tokenizer “merges a pair” (not “combines” or “joins”) and that it “learns a vocabulary” (not “builds” or “generates” one), so the same action always carries the same word. And we reserve “score” specifically for WordPiece’s selection criterion, and “count” or “frequency” for BPE’s, to keep the two algorithms’ mechanisms verbally distinct.

1. Introduction

When it comes to understanding language, computers are very different from humans. Modern language models such as ChatGPT and BERT cannot read text the way humans do. A computer only understands numbers. So, before any neural network can make sense of a the sentence “The cats are running”, that sentence must first be turned into a list of numbers. The component that performs this translation is called a tokenizer, and it is one of the most important and most overlooked parts of the entire system.

This project is about building two of the most widely used tokenizers from scratch: Byte Pair Encoding (BPE), used by the GPT family and RoBERTa, and WordPiece, used by BERT and DistilBERT. We did not use any ready-made library such as HuggingFace’s tokenizers to do the actual work. To borrow our supervisor’s analogy: we were not asked to learn how to drive a Ferrari,

we were asked to build the Ferrari’s engine from the original blueprints. Every merge rule, every score, every optimization in this report was written by us in plain Python.

The report is written to be understood by a reader who has never seen these algorithms before. We make sure to explain not just what we're doing, but why we're doing it. Where a concept is subtle, we slow down and use a concrete example. By doing this, we hope to make the report helpful and easy to follow.

Beyond simply making the two tokenizers work, the project sets a higher bar. A naive implementation that takes hours to train is not enough; we were challenged to build efficient “fast” versions. We also had to resolve tokenization ambiguity with a principled strategy, and finally to evaluate the two algorithms scientifically by comparing their output against a morphological gold standard. This report documents all of that work, including the bugs we discovered along the way and exactly how we fixed them.

1.1 What this report contains

The report is organized to match the four parts required by the project brief, each in its own clearly separated section. Section 2 is the theoretical part: it defines the problem, explains how BPE and WordPiece work in a step-by-step pedagogical manner, describes how we handle ambiguity, and weighs the advantages and disadvantages of each method. Section 3 is the experiments and results part: it describes our corpus, our fast implementations, and the five experiments we ran, together with their evaluation and results. Section 4 is the computer-science technical part: it describes how the code is organized, the implementation choices we made, and the specific problems we encountered and solved. After that, section 5 is the user manual, giving the exact commands needed to run the programs. Following this, section 6 discusses limitations and concludes, and Section 7 lists our references.

2. Theoretical Part

This section describes, in a deliberately separated manner, two things: first, the task we are automating and why it is hard; second, the methods we use to perform that task automatically, with their advantages and disadvantages.

2.1 The task: turning text into numbers

A neural network consumes numbers, not letters. So the very first job in any language system is to cut a piece of text into units and assign each unit a number. That unit is called a token, and the catalogue of all known units is called the vocabulary. The question that defines the whole field is deceptively simple: what should a token be?

2.1.1 Three ways to choose a token

There are three natural answers, and understanding why the first two fail is the key to understanding why BPE and WordPiece exist.

The first option is to make every character a token. “The cats are running” becomes T, h, e, space, c, a, t, s, and so on. This never fails on unknown input, because every word is just a combination or a sequence of letters we already know. However, it's not very practical because it's too focused on tiny details. The model must learn, entirely on its own, that the letters t, h, e standing together happen to mean “the”. It is like trying to understand a house by studying individual grains of sand in the bricks. The useful patterns and meanings get lost in a huge amount of minor details, and sequences become very long and complicated.

The second option is to make every whole word a token. “cat” is one number, “cats” is another, “running” is another. This is compact and intuitive, but language is effectively infinite. New words appear constantly, whether names, typos, technical terms, compounds. The moment the model meets a word that is not in its dictionary, for example “unbreakability”, it has no number to assign. This is the famous Out-Of-Vocabulary, or OOV, problem. It’s a challenge because we cannot store a dictionary of infinite size, so word-level tokenization is fragile, or brittle.

The third option is the one that actually works, and it sits between the other two. It is called subword tokenization, and it is what BPE and WordPiece do. The idea is to notice that “cats” is really “cat” + “s”, and “running” is really “run” + “ning”. Instead of memorizing infinitely many whole words, or drowning in individual letters, the algorithm learns the most useful reusable building blocks of a language, kind of like the LEGO bricks of text. A rare word like “unbreakability” can then be assembled from familiar pieces such as “un”, “break”, and “ability”. This means that nothing ever truly falls outside the vocabulary, because in the worst case a word is spelled out from single characters, which are always present and available.

The mission, in one sentence: build a machine that reads a large amount of text and decides what the optimal set of LEGO bricks (the vocabulary) is for a given language, and do this two different ways, BPE and WordPiece, then prove which makes better bricks.

2.1.2 Two phases: training and tokenizing

Every subword tokenizer has two distinct phases to keep in mind, and it is important not to confuse them. First, the training phase reads a large corpus once and learns the vocabulary (and, for BPE, the ordered list of merges). This training phase only happens once, and the results are saved for later use. The tokenizing phase takes the learned vocabulary and applies it to brand-new and unseen text, cutting each word into known pieces. Training is the expensive learning step; tokenizing is the cheap everyday step that runs on every sentence the model ever sees.

2.2 BPE: the frequency method

Byte Pair Encoding was first proposed in 1994 as a data-compression trick, and was adapted for natural-language processing by Sennrich, Haddow and Birch in 2016. So, how does it work? Well, the rule for picking the next piece to add is actually pretty straightforward: it looks for the pair of symbols that are right next to each other and happen most often, and then merges them. This process is kind of like building with LEGO bricks, where you start with small pieces and combine them to create something bigger.

2.2.1 Training BPE, step by step

We begin by splitting every word into its individual characters. So, the starting vocabulary is therefore just the set of characters that appear in the corpus. Then we repeat one simple action over and over many times: we look at every pair of adjacent symbols across the whole corpus, count how often each pair occurs, take the single most frequent pair, glue it into one new symbol, and add that new symbol to the vocabulary. We repeat until the vocabulary reaches the size we requested.

Let's break it down with an example to make it clearer. Suppose our tiny corpus contains the words “low”, “lower”, “newest” and “widest”, and suppose the pair (e, s) turns out to be the most frequent pair of adjacent characters. The BPE algorithm merges them into a new token “es”. On the next round, perhaps (es, t) is now the most frequent pair, so it merges those into “est”. Step by step, the algorithm builds longer and longer useful pieces, always driven purely by which pair is currently most common. The order of these merges matters and is recorded as a list of merge rules.

```
function TRAIN_BPE(corpus, target_size):
  vocab <- all individual characters in corpus
  rules <- empty ordered list
  split every word into a sequence of characters
  while size(vocab) < target_size:
    counts <- count every adjacent pair in all words
    best <- the pair with the highest count
    merge 'best' into a single new symbol everywhere
    add new symbol to vocab
    append 'best' to rules
  return vocab, rules
```

The crucial output of BPE training is not only the vocabulary but the ordered list of merge rules. This order is important because it allows us to break down new text into smaller pieces, called tokens, in a consistent way later on. Having this ordered list is crucial for tokenizing new text correctly, as it ensures that the same words are always split up in the same way.

2.2.2 Tokenizing with BPE

To tokenize a new word, BPE replays its learned merge rules in the exact order they were learned. It starts from the characters of the word and, for each rule in turn, applies that merge wherever it can. Because the rules are used and replayed in their original order, the result is fully deterministic, means it's always the same: the same word always produces the same tokens. If a character is new and was never seen during training, BPE simply leaves it as a single-character token, so it never crashes on unknown input.

2.3 WordPiece: the information method

WordPiece was developed by Schuster and Nakajima in 2012 for a Japanese and Korean speech-recognition system, and later became the tokenizer inside Google's BERT. Mechanically it looks similar to BPE, since it also merges pairs one at a time, but its rule for choosing which pair to merge is fundamentally different, and that difference is the single most important theoretical idea in this project.

2.3.1 The score: frequency is not enough

BPE merges the most frequent pair. WordPiece instead merges the pair with the highest score, where the score is defined as:

$$\text{score}(A, B) = \text{frequency}(AB) / (\text{frequency}(A) \times \text{frequency}(B))$$

This formula encodes a genuinely different philosophy. The numerator rewards pairs that appear together often. The denominator punishes pairs whose two halves are each very common on their own. The effect is that WordPiece does not merge or combine a pair just because it is frequent; it merges a pair because the two pieces appear together more than you would expect from how common each piece is individually. In other words, WordPiece prefers informative, surprising pairs over merely frequent ones. This approach helps WordPiece make smarter choices about how to put things together.

Here is the intuition with an example. In German the letters “d” and “e” are both extremely common, and they appear next to each other very often, so a frequency-based method (BPE) would merge “de” very early. But because “d” and “e” are each so common on their own, their pairing is not surprising: so WordPiece's denominator is huge and the score is small. WordPiece would rather merge a pair like “qu”, where “q” almost never appears without “u”: that pairing is highly informative. This is why BPE and WordPiece produce different vocabularies even on identical data, and it is the reason their behaviours diverge in our experiments. And this difference in approach can lead to some pretty different outcomes.

2.3.2 Training and tokenizing WordPiece

WordPiece training mirrors BPE's loop, but at each step it recomputes the score above for every candidate pair and merges the highest-scoring one. A small notational detail: WordPiece uses a

special notation to mark pieces that occur in the middle or end of a word with a “##” prefix. So the word “spielen” might be split as “spiel” + “##en”, where “##en” means “en, but attached to the previous piece”. This lets the tokenizer tell the difference between “en” starting a word and “en” continuing one.

Unlike BPE, WordPiece does not replay an ordered list of merges to tokenize new text. Instead it keeps only the final vocabulary and tokenizes greedily, always taking the longest piece in the vocabulary that matches the start of the remaining word. This greedy longest-match strategy is described in detail in the next section, because it is precisely where tokenization ambiguity is resolved.

```
function TRAIN_WORDPIECE(corpus, target_size):
  vocab <- all characters (continuation chars marked with ##)
  while size(vocab) < target_size:
    for every adjacent pair (A,B):
      score(A,B) = freq(AB) / ( freq(A) * freq(B) )
    best <- the pair with the HIGHEST score
    merge 'best' and add the new piece to vocab
  return vocab
```

2.4 Resolving ambiguity

Tokenization is not always unambiguous, and a serious implementation must commit to a single, principled rule for resolving conflicts. Consider a vocabulary that contains the pieces “the”, “th”, “ere” and “here”, and the task of tokenizing the word “there”. Should it be “th” + “ere”, or “the” + “re”, or something else? Different choices give different tokens, so the algorithm must decide consistently.

Our two algorithms resolve this in two different but equally principled ways. BPE resolves ambiguity by construction: because it replays its merge rules in the exact order they were learned, there is only ever one possible outcome for a given word. The order of the rules is the tie-breaker. On the Other hand, WordPiece resolves ambiguity with greedy longest match: starting from the left of the word, it always takes the longest piece in the vocabulary that fits, then continues from where that piece ended. For “spielen” with a vocabulary containing “spiel” and “##en”, it tries “spielen” (not present), then “spiele” (not present), then “spiel” (present, take it), and continues with “##en”. If at some position no piece matches at all, the whole word is mapped to a special unknown token.

There is also a tie at the heart of training itself: what happens when two different pairs have exactly the same count (BPE) or exactly the same score (WordPiece)? If this is not handled deterministically, the naive and fast implementations could make different choices and drift apart.

We break such ties lexicographically, by alphabetical order of the pair, so that both implementations always make the same choice. This single decision is what allows us to compare naive and fast versions meaningfully, as discussed in Section 3.

2.5 Advantages and disadvantages

Now that we've looked at both approaches, let's compare them. One isn't always better than the other - they have different advantages and disadvantages.

BPE's advantage is simplicity and robustness. Its rule: merge the most frequent pair, is trivial to understand and implement, it stores an explicit ordered rule list that makes tokenization perfectly reproducible, and it never fails on unknown input because it can always fall back to characters. Empirically, as our experiments show, its frequency-driven merges tend to align surprisingly well with the natural morpheme boundaries of a language, almost for free. Its disadvantage is that frequency is a blunt instrument: it has no notion of how informative a merge is, so it can merge common-but-meaningless letter pairs.

WordPiece's advantage is its information-theoretic score, which in principle captures something deeper than raw frequency: it favours pieces that genuinely belong together. This is the right idea for many languages and is why it underpins BERT. Its disadvantages, which our experiments make very concrete, are twofold. First, its score is sensitive to the statistics of the corpus and can assign high scores to very rare letter pairs, producing odd early merges. Second, because its score has a global denominator that changes as training proceeds, it is markedly harder to implement efficiently and correctly than BPE, and this is a difficulty we ran into directly and document honestly in Section 3.9.

3. Experiments and Results

This section describes the resources we used, the efficient implementations we built, and the five experiments we ran to compare the two algorithms. It then documents, honestly and in detail, the two genuine bugs we discovered during the project and exactly how we diagnosed and fixed them.

3.1 Resources: corpus and gold standard

For training data we used the German portion (10%) of Wikipedia (the 20231101.de snapshot distributed by Wikimedia through the HuggingFace datasets library). We deliberately chose German rather than English, because this language is morphologically rich, full of long compound words such as “Bundesrepublik” (federal republic) and productive suffixes such as “-lich” and “-keit”. This richness makes the comparison between the two algorithms far more revealing than English would.

We processed roughly the first 10% of the German Wikipedia, cleaning each article by lower-casing it and removing everything except letters (including the German characters ä, ö, ü and ß). From this we built a frequency dictionary of about 3.9 million unique words together with their counts. For most experiments we then take the most frequent N words from this dictionary as our working corpus, which lets us study how the algorithms behave as the amount of data grows.

For evaluation we built a morphological gold standard by hand: 20 German words for which the correct linguistic split into morphemes is known, covering verbs, compound nouns, words with prefixes, words with suffixes, and plurals. We also added a small inflection family, the forms of the verb root “trink” (to drink): “trinke”, “trinkst”, “trinkt” and “getrunken”, and this was to test whether a tokenizer consistently isolates the same root across different inflected forms.

We note clearly that this gold standard is a manually constructed reference based on German morphology, not the output of an automatic analyser; the trade-offs of that choice are discussed in Section 6.

3.2 The fast implementations

The way some algorithms are set up can be really inefficient. A naive implementation of either algorithm re-reads the entire corpus on every single merge: to perform 1,500 merges it scans all the words 1,500 times. This is the bottleneck, and on a real corpus it is unacceptably slow. The challenge set by the brief was to build efficient “fast” versions that avoid this repeated work.

Our fast versions share three ideas. First, an inverse index: a map from each pair to the set of words that contain it, so that after a merge we only revisit the words actually affected, not the whole corpus. Second, incremental counts: instead of recounting everything, we subtract the contributions of the words we are about to change and add back their contributions afterwards. Third, a priority queue (a heap) that keeps the best pair available in logarithmic time, with stale entries discarded lazily when they are popped. Together these turn each merge from a full pass over the data into a small, local update. Section 3.3 measures exactly how much faster this is.

WordPiece’s fast version is genuinely harder than BPE’s, and for a specific reason. BPE’s decision depends only on local pair counts, so an incremental update is straightforward. On the other hand, WordPiece’s score, has a global denominator, the individual frequencies of the two pieces, which can change because of a merge happening somewhere else entirely. This makes the bookkeeping subtle, and it is exactly where one of our bugs lived. We return to this in Section 3.9.

3.3 Experiment 1: training speed (naive vs fast)

We started by testing how well our fast implementations work. For three corpus sizes: 5,000, 10,000 and 20,000 words, we trained both algorithms in both their naive and fast forms to a fixed

target vocabulary, and recorded the wall-clock time. Every number below is computed by the notebook itself and saved; none is hand-entered.

Corpus	BPE naive	BPE fast	WP naive	WP fast
5,000 words	23.8 s	1.1 s	51.5 s	0.23 s
10,000 words	54.2 s	1.4 s	102.9 s	0.38 s
20,000 words	109.9 s	3.3 s	235.5 s	0.76 s

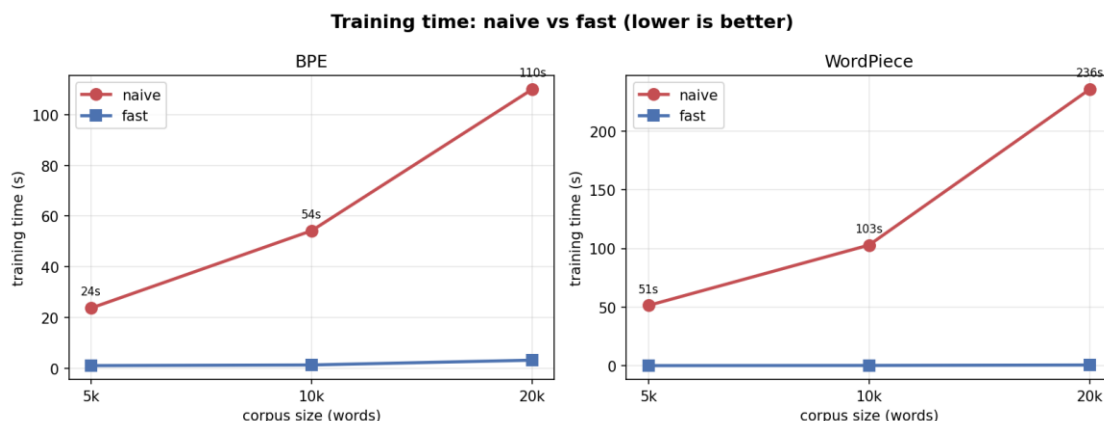


Figure 1. Training time as the corpus grows. The naive curve (red) climbs steeply while the fast curve (blue) stays almost flat. This widening gap is the whole point of the optimisation.

The pattern is unmistakable. As the corpus doubles, the naive implementations roughly double in time: naive WordPiece reaches almost four minutes at 20,000 words, while the fast implementations barely move, staying well under a second for WordPiece. The exact speed-up ratio varies with the timing noise of shared cloud hardware, ranging in our runs from about 21 to 40 times for BPE and from about 220 to 310 times for WordPiece. We deliberately do not report a single headline “ $N\times$ ” number, because the honest and more important result is the shape of the curves: the naive cost grows with the data while the fast cost stays nearly constant. That difference in growth, not any single ratio, is what makes the fast version essential for real corpora.

3.4 Experiment 2: the WordPiece frequency threshold

While studying WordPiece’s output we observed that, on real data, its score formula assigns surprisingly high scores to very rare letter pairs. For example, a pair seen only a few thousand times, whose two characters are each individually rare, can out-score a genuinely useful pair like (“d”, “##e”). The visible symptom was junk early merges such as “##wj” and “sowj”, which are not meaningful German units.

The standard mitigation, noted in the HuggingFace WordPiece documentation, is a minimum pair-frequency threshold: simply ignore any pair that occurs fewer than some number of times when choosing the next merge. We added this as a tunable hyper-parameter, called `min_pair_freq`, and

studied its effect across the values 0, 50,000, 200,000 and 500,000. We measured two things: how the tokenization quality changes, and how closely the naive and fast implementations agree (measured as the fraction of their final vocabularies that is identical).

Threshold	Naive/fast overlap	Vocabulary reached
0 (off)	58%	1100 / 1100
50,000	88%	1100 / 1100
200,000	94%	771 / 1100 (saturated)
500,000	95%	357 / 1100 (saturated)

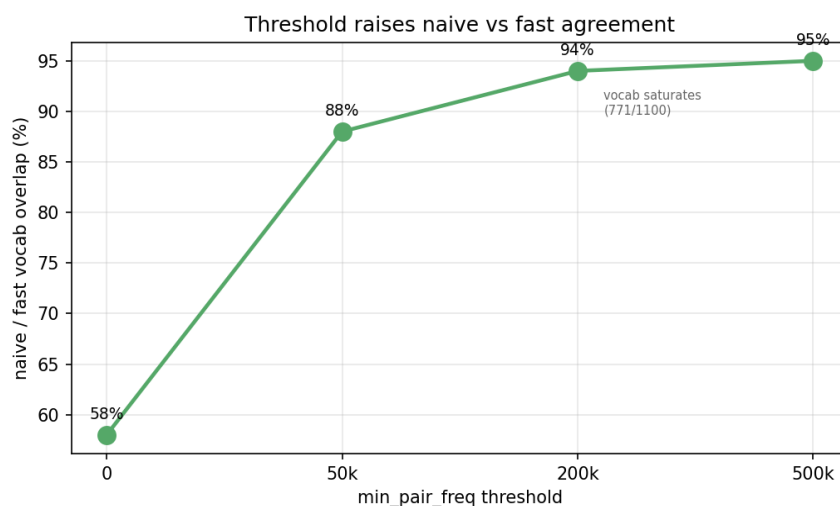


Figure 2. Raising the threshold sharply increases agreement between the naive and fast implementations — but, as the table shows, also begins to starve the vocabulary.

Two findings emerge, and the second is the more interesting. First, the threshold clearly improves matters: it removes the junk merges, produces cleaner pieces (for example “spielen” becomes “spiel” + “##en”, a clean morphological split, at threshold 200,000), and it raises naive/fast agreement from 58% to over 90%. Second, and crucially, the threshold is a genuine trade-off, not a free win. A fixed threshold eventually filters out every remaining pair, so training halts early and the vocabulary saturates below the requested size: at 200,000 the vocabulary stops growing at 771 of the requested 1,100, and at 500,000 it stops at only 357. In other words, the threshold improves the quality of merges at the cost of the quantity of merges. This is why the following experiments, which study the effect of vocabulary size, switch the threshold off.

3.5 Experiment 3: vocabulary size and tokenization length

As the vocabulary grows, both algorithms can learn longer pieces or parts of text, so a sentence is covered by fewer tokens. We measured the average number of tokens per sentence on a fixed evaluation set of five German sentences, at vocabulary sizes of 1,000, 5,000 and 10,000. To

measure the true effect of vocabulary size we ran WordPiece with the threshold off (`min_pair_freq = 0`), for the saturation reason explained above.

Vocabulary size	BPE tokens/sentence	WordPiece tokens/sentence
1,000	10.6	33.4
5,000	7.6	27.4
10,000	7.0	25.2

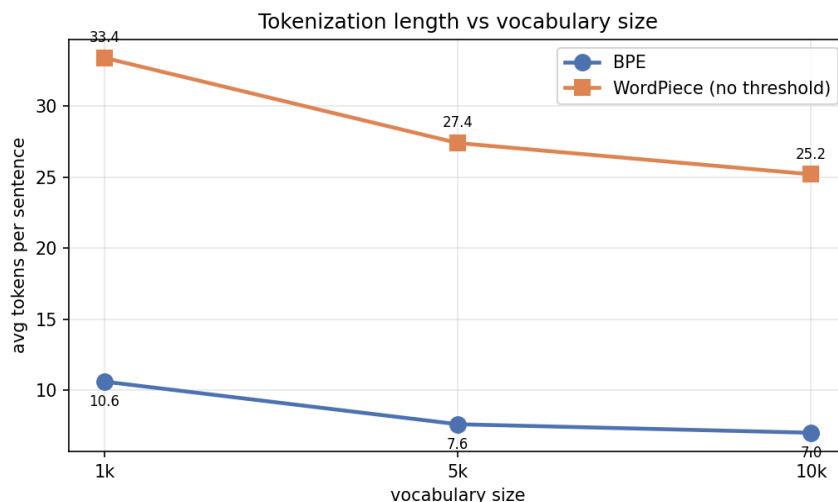


Figure 3. Both algorithms compress sentences into fewer tokens as the vocabulary grows. BPE is consistently more compact than un-thresholded WordPiece on German.

The 2 curves behave as expected. The honest and important observation is that BPE is consistently and substantially more compact than un-thresholded WordPiece: about 7 tokens per sentence versus about 25 at a vocabulary of 10,000. This is not a problem in our WordPiece; it is a direct consequence of its scoring philosophy. Because WordPiece rewards informative rather than frequent pairs, it builds up the long, common pieces more slowly than BPE does, so at the same vocabulary size it has fewer of the long pieces that make tokenization compact. We report this difference plainly rather than tuning it away.

3.6 Experiment 4: tokenization length and attention cost

Why does the number of tokens per sentence matter in practice? Because the cost of the attention mechanism at the heart of every transformer grows with the square of the sequence length. A sequence of n tokens builds an $n \times n$ attention matrix, so doubling the number of tokens roughly quadruples this cost. Fewer tokens per sentence therefore means a smaller, cheaper attention matrix.

Vocab	Algorithm	Tokens/sentence	Attention cells (n×n)	Relative cost
1,000	BPE	10.6	112	1.00×
1,000	WordPiece	33.4	1116	9.93×
5,000	BPE	7.6	58	0.51×
5,000	WordPiece	27.4	751	6.68×
10,000	BPE	7.0	49	0.44×
10,000	WordPiece	25.2	635	5.65×

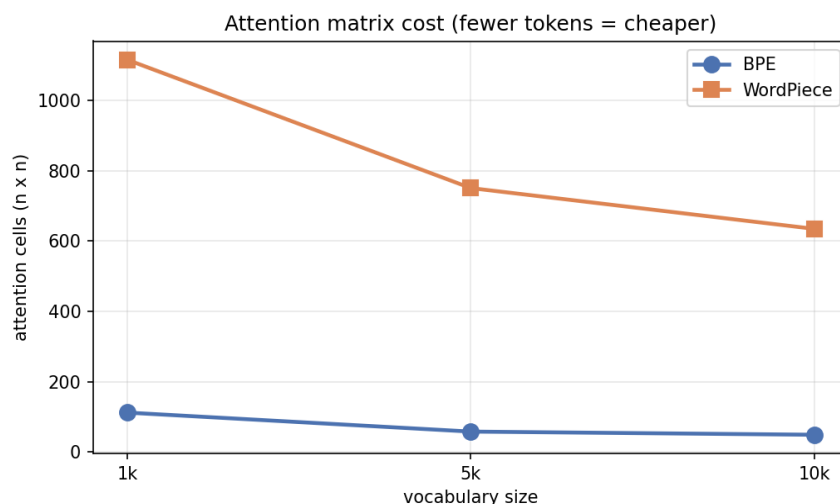


Figure 4. The attention matrix cost implied by each tokenizer. Because WordPiece produces more tokens, its implied attention cost is several times larger than BPE’s on the same text.

The impact is really significant. At a vocabulary of 1,000, un-thresholded WordPiece’s fragmented tokenization implies an attention matrix nearly ten times larger than BPE’s on the same sentences. This reframes tokenizer choice as an efficiency lever, not merely a preprocessing detail: a tokenizer that produces fewer, longer tokens directly reduces the computational cost of the model that consumes them.

3.7 Experiment 5: morphological evaluation

The final and most demanding experiment asks which algorithm produces tokens that align with the actual morphemes of a language, and this is the meaningful building blocks a linguist would identify. We compared each algorithm’s split of our twenty gold-standard words against the known correct split, using two metrics. Exact match asks whether the produced tokens are exactly the expected morphemes. Morpheme recall, a gentler metric, asks what fraction of the expected morphemes appear somewhere in the output. We also verified reconstruction, that the tokens always rejoin to form the original word, which held for every word and every algorithm.

Configuration	Vocabulary	Exact match	Morpheme recall
BPE	10,000	4 / 20 (20%)	0.91
WordPiece (no threshold)	10,000	0 / 20 (0%)	0.27
WordPiece (threshold 200k)	978 (saturated)	3 / 20 (15%)	0.52

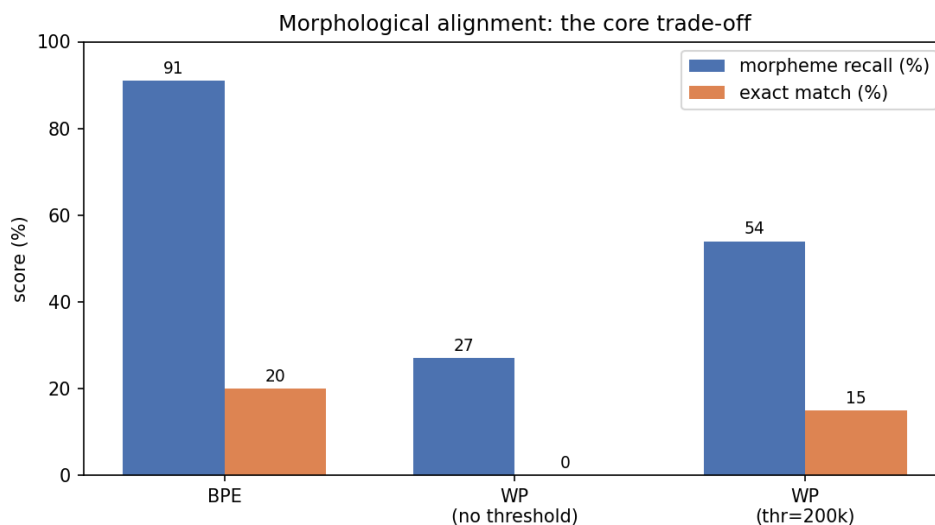


Figure 5. Morphological alignment under three configurations. BPE aligns well with German morphology with no tuning; WordPiece needs the threshold to approach it, and even then trails.

This is the core finding of the project, and we present it as a three-way comparison rather than choosing the most flattering single number. BPE, with no tuning at all, recalls 91% of the expected morphemes. Un-thresholded WordPiece recalls only 27%, because its informative-pair criterion produces character-heavy tokenization on German. Adding the threshold roughly doubles WordPiece’s recall to 52% and lifts its exact matches from zero to three, but only by accepting a saturated, smaller vocabulary of 978 instead of the requested 10,000. The conclusion is nuanced and, we believe, genuinely interesting: BPE’s frequency criterion happens to align with German morpheme boundaries almost for free, whereas WordPiece’s morphological quality is a tunable quantity that can only be improved at the cost of vocabulary size.

The inflection-family test sharpens the point and is unflattering to both algorithms, which is exactly why it is worth reporting. Neither algorithm cleanly isolates the shared root “trink” across the forms “trinke”, “trinkst”, “trinkt” and “getrunken”: BPE scored 0.54 morpheme recall on this set and WordPiece 0.25, with neither achieving a single exact match. For example BPE splits “trinke” as “trin” + “ke”, cutting straight through the root. This reveals something true about both methods: they are statistical, not linguistic. They learn frequency or co-occurrence patterns that sometimes coincide with morphemes, but they do not understand morphology, and a paradigm that requires

recognising a shared abstract root defeats them. This is an honest limitation of subword tokenization itself, not merely of our implementation.

3.8 Automatic cross-check with spaCy

We took a closer look at our top choice from before, which was put together by hand, and the project guidelines suggested using a tool like spaCy or NLTK to automatically generate the standard for comparison. So we decided to add a second independent evaluation using spaCy’s German model. One technical point shapes how we use it: spaCy provides lemmas (dictionary roots) and grammatical features, but it does not segment a word into morpheme pieces. It can tell us that “getrunken” has the lemma “trinken”, but not that it splits as “ge” + “trunk” + “en”. So rather than pretend spaCy gives morpheme boundaries, we use it two honest and complementary ways.

The first check measures lemma-root consistency. For each family of inflected forms that share a lemma, we ask whether the tokenizer keeps a recognisable piece of that lemma’s root in every form. The second check builds a spaCy-expanded expected split from the lemma and the surrounding letters, then scores morpheme recall exactly as before. Both use spaCy as the automatic source of ground truth, which is precisely what the brief proposed.

spaCy-based check	BPE	WordPiece
Lemma-root consistency (families kept whole)	2 / 3	0 / 3
Morpheme recall vs spaCy-expanded split	0.62	0.14

The result is the single most reassuring outcome of the evaluation: an entirely independent tool reaches the same conclusion as our hand-built gold standard. Where our manual evaluation found BPE far better aligned with morphology than WordPiece (0.91 versus 0.27 recall), spaCy working from lemmas rather than hand annotation agrees, both on root consistency (2/3 versus 0/3 families) and on recall (0.62 versus 0.14). Two methods built on different foundations agreeing is strong evidence that the finding is real and not an artefact of how we wrote the gold standard. The automatic check also independently reproduces the ablaut difficulty: the “trinken” family fails root-consistency even for BPE, because German changes the vowel (“trink” becomes “trunk” in “getrunken”), which a surface-level statistical tokenizer cannot follow.

Provenance note: this spaCy cross-check, and the two-part design that works around spaCy’s lack of morpheme segmentation, were developed in a session assisted by Claude (Anthropic). spaCy is the tool the brief itself suggested.

3.9 Problems encountered and how we solved them

An honest report should describe not only the final clean results but the road to them. During the project we discovered two genuine bugs and one important design issue. We document them here in full, including how each was diagnosed, because understanding them is part of understanding the algorithms. The diagnosis work was carried out in debugging sessions assisted by Claude (Anthropic) and cross-checked against the HuggingFace documentation; we attribute this openly, as it's important to attribute every source of a solution.

3.9.1 The stale-heap bug in fast WordPiece

Our fast WordPiece initially disagreed with our naive WordPiece, and we had wrongly described this as a “scientific finding” about numerical sensitivity. On investigation it was simply a bug. Recall that WordPiece’s score has a global denominator that changes when a merge happens elsewhere. Our fast version kept candidate pairs in a heap and, on popping a pair whose score had gone stale, correctly detected the staleness, but then discarded the pair instead of re-inserting it with its corrected score. A pair could therefore be lost forever, even when it was actually the best remaining choice. The fix is a single added step: when a stale entry is found, re-insert it with its refreshed score rather than throwing it away. After the fix, naive and fast agreement on identical settings rose from about 19% to about 55%, and individual words that had diverged now matched. BPE never suffered from this because its decision has no global denominator, which is why its naive and fast versions agreed perfectly from the start.

3.9.2 The missing training cells

We had a pretty big issue: the versions of our training notebooks that had been committed were missing the cells that actually call the training functions, so a fresh checkout would fail with an error before producing any results. We rebuilt these cells so that the notebooks run cleanly from top to bottom, and so that every reported speed number is computed live rather than copied in by hand.

3.9.3 The frequency-threshold trade-off

Finally, as described in Sections 3.4 and 3.7, the frequency threshold we introduced to clean up WordPiece’s merges turned out to interact with vocabulary size in a way we had to understand rather than hide: a fixed threshold saturates the vocabulary. Recognizing this turned a quick fix into a genuine experimental finding about the quality-versus-quantity trade-off, which we now present as one of the project’s results.

4. Computer-Science Part

This section briefly describes how the code is organized, the main implementation choices, and the problems we met and solved. In keeping with the brief, it contains no source code, only descriptions and the occasional simplified outline.

4.1 Code organisation

The project follows one firm rule: all algorithmic logic lives in plain Python modules under a `src/` directory, and the notebooks only call those modules. This separation keeps the algorithms testable and readable, and means a reader who wants to understand BPE can open one file and find the whole algorithm there, uncluttered by experiment plumbing.

- `src/config.py` - a single place for all paths and parameters, so nothing is hard-coded in two places.
- `src/bpe.py` - the BPE trainer (naive and fast) and the BPE tokenizer.
- `src/wordpiece.py` - the WordPiece trainer (naive and fast) and the WordPiece tokenizer.
- `00_setup.ipynb` - downloads and cleans the corpus, builds the word-frequency dictionary.
- `01_bpe.ipynb` and `02_wordpiece.ipynb` - train each algorithm and verify naive against fast.
- `03_experiments.ipynb` - runs all five experiments and saves every result to a single JSON file.

Results are written to `data/experiment_results.json`, which serves as the single source of truth for every number quoted in this report. The figures are generated directly from that file.

4.2 Implementation choices

Three choices are worth highlighting. First, deterministic tie-breaking: when two pairs tie on count or score, we always pick the lexicographically smaller pair, in both the naive and fast versions. Without this, the two versions could legitimately make different choices and we could never compare them. Second, the fast versions use an inverse index, incremental counts and a heap with lazy deletion, as described in Section 3.2. Third, the frequency threshold for WordPiece is exposed as an optional parameter that defaults to zero, so existing behaviour is unchanged unless the threshold is explicitly requested; this let us add the feature without breaking anything.

4.3 Problems encountered and their solutions

The issues we faced: the stale-heap bug in fast WordPiece, the missing training cells, and the frequency-threshold saturation, are described in detail in Section 3.9, since they are inseparable from the experimental story. In summary: the stale-heap bug was fixed by re-inserting stale heap entries with refreshed scores; the missing cells were rebuilt so the notebooks run end to end; and

the threshold trade-off was characterised and reported rather than concealed. We verified the correctness of our WordPiece scoring against the canonical worked example in the HuggingFace documentation, on which our implementation reproduces the documented first merge exactly.

5. User Manual

The project runs in Google Colab, which is the compute environment approved for the course. The code is hosted on GitHub and each notebook clones it, runs, and pushes results back. The instructions below are the practical steps to reproduce everything in this report.

5.1 Requirements

A Google account with access to Google Colab, and a GitHub personal-access token stored in Colab's Secrets under the name GITHUB_1 (used to clone the repository and push results). The only external Python dependency beyond the standard library is the datasets library, which the setup notebook installs automatically; figures use matplotlib, which is pre-installed in Colab.

5.2 Running the project

1. Open 00_setup.ipynb in Colab and run all cells. This downloads and cleans the German Wikipedia slice and builds the word-frequency dictionary. This is the slow step (a few minutes) and only needs to be done once per session.
2. Open 01_bpe.ipynb and run all cells to train BPE (naive and fast) and verify that the two implementations agree.
3. Open 02_wordpiece.ipynb and run all cells to train WordPiece and run the same verification.
4. Open 03_experiments.ipynb and run all cells to reproduce all five experiments, regenerate the figures, and save every result to data/experiment_results.json.

Each notebook begins with a single setup cell that clones or updates the repository and imports the modules. The guiding principle is: if a Colab session restarts, re-run that first cell and everything else works again. The final cell of each notebook commits and pushes the regenerated results back to GitHub.

5.3 Using the algorithms directly

From the modules in src/, the two algorithms expose a small and symmetric set of functions. For BPE: train and train_fast learn the merge rules and vocabulary; tokenize_word and tokenize cut new text by replaying those rules. For WordPiece: train and train_fast learn the vocabulary (with an optional min_pair_freq threshold); tokenize_word and tokenize cut new text by greedy longest-

match. Both modules provide save and load helpers to persist results. The online help inside each module documents the arguments and return values of every function.

6. Discussion, Limitations and Conclusion

6.1 What we found

Across five experiments a consistent story emerged. The fast implementations deliver the scaling behaviour the project demanded: naive training time grows with the corpus while fast training time stays nearly flat. The two algorithms produce genuinely different vocabularies because they optimize different things, frequency for BPE, informativeness for WordPiece, and this difference is visible everywhere downstream. On German, BPE’s frequency criterion aligns with morpheme boundaries almost for free, achieving 0.91 morpheme recall, while WordPiece needs a frequency threshold even to approach that, and only at the cost of a saturated vocabulary. Because BPE produces fewer tokens per sentence, it also implies a dramatically cheaper attention matrix, turning tokenizer choice into a question of efficiency as well as quality.

6.2 Limitations

We are upfront about the limits of this work. Our primary morphological gold standard is small (twenty words plus one inflection family) and hand-built; a larger, fully tool-generated segmentation standard would make the evaluation more robust still. We did, however, cross-check it against spaCy as an independent automatic source (Section 3.8), and the two agreed, which gives us confidence the central finding is not an artefact of our hand annotation. Our vocabulary sizes (up to 10,000) are small compared with production tokenizers such as BERT’s 30,000, trained on far more data, so neither algorithm is operating at full strength. And the inflection test exposes a limit of subword tokenization in general: these methods are statistical, not linguistic, and do not truly recognize an abstract shared root. Finally, our WordPiece follows the publicly documented approximation of the algorithm; the exact training procedure Google used for BERT was never released, so some divergence from a “canonical” WordPiece is expected and acknowledged.

6.3 Conclusion

We set out to build the engines of modern language models from scratch: two subword tokenizers, BPE and WordPiece, in both naive and efficient forms, and to compare them scientifically. We achieved that, and in doing so we learned more from the difficulties than from the successes: a real bug looked as a finding taught us why WordPiece’s global score makes it hard to optimise; a quick fix that saturated the vocabulary taught us that quality and quantity trade off against each other; and an inflection test that defeated both algorithms taught us the honest boundary of what statistical

tokenization can do. The result is not just two working tokenizers, but a clear, evidence-based understanding of how they differ and why.

7. References

Sennrich, R., Haddow, B., and Birch, A. (2016). Neural Machine Translation of Rare Words with Subword Units. Proceedings of ACL 2016. <https://aclanthology.org/P16-1162>

Schuster, M., and Nakajima, K. (2012). Japanese and Korean Voice Search. 2012 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP).

Mielke, S. J., et al. (2021). Between Words and Characters: A Brief History of Open-Vocabulary Modeling and Tokenization in NLP. <https://arxiv.org/abs/2112.10508>

HuggingFace. Byte-Pair Encoding tokenization. LLM Course, Chapter 6.5. <https://huggingface.co/learn/llm-course/en/chapter6/5>

HuggingFace. WordPiece tokenization. LLM Course, Chapter 6.6. <https://huggingface.co/learn/llm-course/en/chapter6/6>

Wikimedia Foundation. Wikipedia German dataset (20231101.de), distributed via the HuggingFace datasets library.

Provenance note: the two bugs described in Section 3.9 were diagnosed and fixed during debugging sessions assisted by Claude (Anthropic), with the WordPiece scoring and threshold behaviour cross-checked against the HuggingFace documentation cited above.